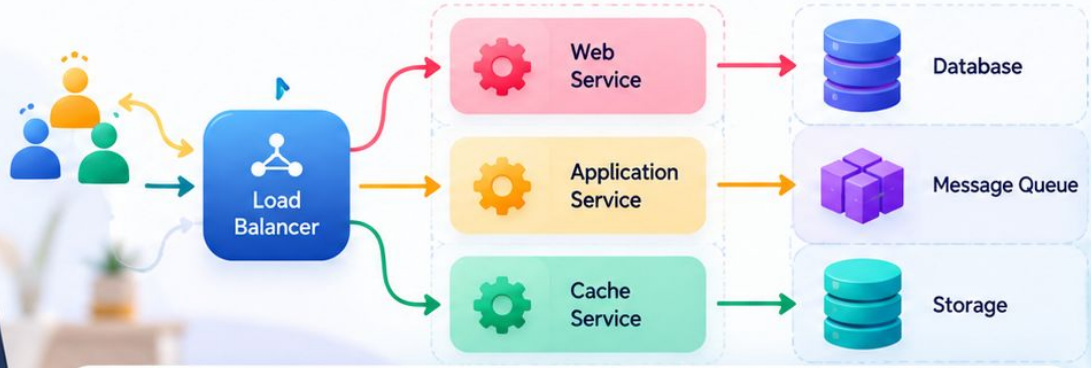
Introduction to System Performance

Performance - Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews









Performance | Concepts | Tools & Techniques | System Design for Interviews

Instructor: Rahul Rajat Singh

What is Performance in System Design?

Performance = How efficiently a system meets its functional requirements under load.

Key Dimensions:

Speed
Time to respond

Capacity
Amount of work handled

Efficiency
Resource usage under load

Performance is not a single metric — it's a **multi-dimensional goal.**



Latency vs. Throughput



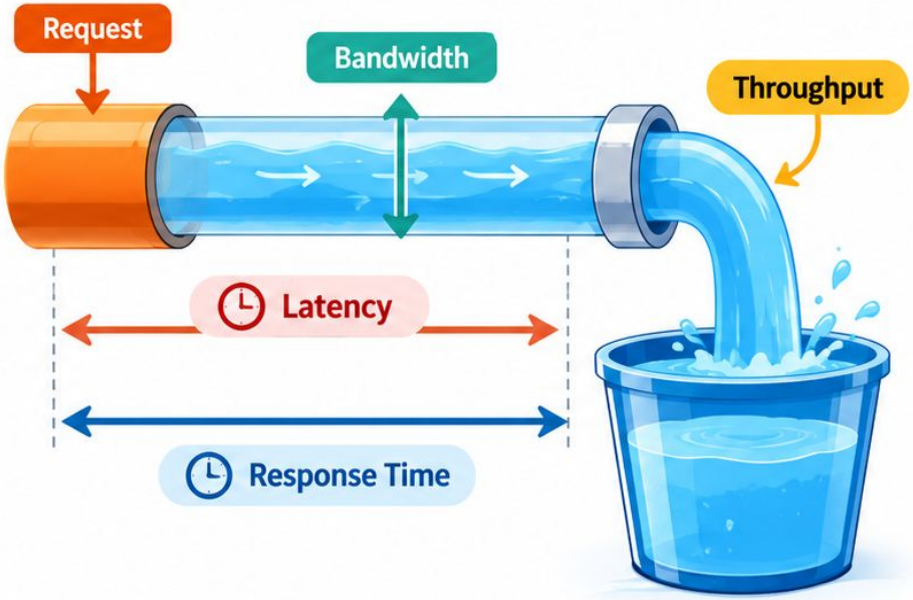
Latency

- Time taken to process one request
- Measured in ms or s
- Affects responsiveness



Throughput

- Number of requests processed per second
- Measured in RPS or TPS
- Affects scalability





Low latency ≠ High throughput and vice versa.



Both must be balanced based on use case.

Instructor: Rahul Rajat Singh

Scalability vs. Responsiveness

Scalability

Ability to handle increased load without performance degradation.

Horizontal vs. Vertical scaling

Responsiveness

System's ability to respond quickly.

Tightly linked to latency

Key Takeaway

Good design should ensure responsiveness at scale.



Measuring Performance

Performance must be measurable & trackable.

SLA | **Service Level Agreement**
External contractual guarantee.

SLO | **Service Level Objective**
Internal target.

SLI | **Service Level Indicator**
Actual metric value.



Example

1. **SLA:** 99.9% uptime
2. **SLO:** 95% of requests < 300ms
3. **SLI:** Actual measurement (e.g., 93% of requests < 300ms)

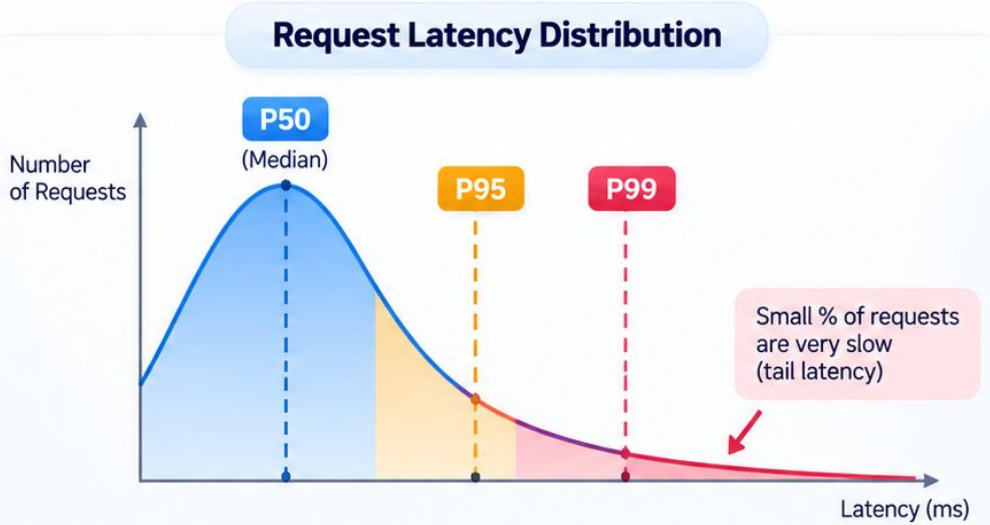
Understanding Percentiles

Mean/average ≠ useful in tail-latency-sensitive systems.

Percentiles provide better insight:

- **P50**: Median
- **P95**: 95% of requests are faster than this
- **P99**: Tail latency — critical for user experience

Track tail latencies for real-world performance insights.



1. **50% of requests** ≤ P50
2. **45% of requests** P50 → P95
3. **4% of requests** P95 → P99
4. **1% slowest** Beyond P99

Why Performance Matters in Modern Applications

Users expect near-instant responses
across mobile and web.

Poor performance leads to:

- Drop-offs and bounce rates
- Lost revenue
- System instability under load

Performance is a feature,
not an afterthought.

It shapes user experience,
operating cost, and reputation.



PERFORMANCE

- User Experience
- Business Growth
- System Stability
- Cost Optimization
- Reputation

Instructor: Rahul Rajat Singh

Performance Testing Overview



Load Testing



Tests system performance under **normal load conditions.**

Stress Testing



Tests system behavior **beyond normal limits.**

Spike Testing



Tests system response to a **sudden large load.**

Endurance Testing



Tests system performance over an **extended time period.**

Performance Testing

- **Load Test** — Normal Traffic
- **Stress Test** — Beyond Capacity
- **Spike Test** — Sudden Traffic Burst
- **Endurance Test** — Long Duration Load



Goals:

1. Identify bottlenecks
2. Ensure reliability under real-world scenarios



Instructor: Rahul Rajat Singh

Introduction to Performance Monitoring

1 Monitoring ≠ Testing

Monitoring is continuous—
not a one-time test.

2 Key Tools

APM
New Relic,
Datadog

Logs & Metrics
ELK, Prometheus
+ Grafana

3 What to Track

- Latency & throughput
- Error rates
- Resource usage (CPU, memory, DB queries)



Instructor: Rahul Rajat Singh

Interview Questions — System Performance



1. 

   What is the difference between latency and throughput?

2. 

   How do SLAs, SLOs, and SLIs differ? Provide real-world examples.

3. 

   Why are percentiles (like P95, P99) important in performance monitoring?

4. 

   What strategies would you use to identify a system's performance bottleneck?

5. 

   How would you ensure responsiveness in a highly scalable system?

6. 

   What tools or techniques have you used for performance testing and monitoring?

7. 

   How would you design a system to handle sudden traffic spikes?

8. 

   Explain the trade-offs between performance and cost in cloud environments.

Summary and Key Takeaways

01



Performance = speed, efficiency & scalability under load.

02



Understand latency vs. throughput.

03



Set and measure SLAs, SLOs, and percentiles.

04



Performance must be tested, monitored, and prioritized.

05



Essential for modern, scalable, and user-friendly systems.

WHAT'S NEXT →

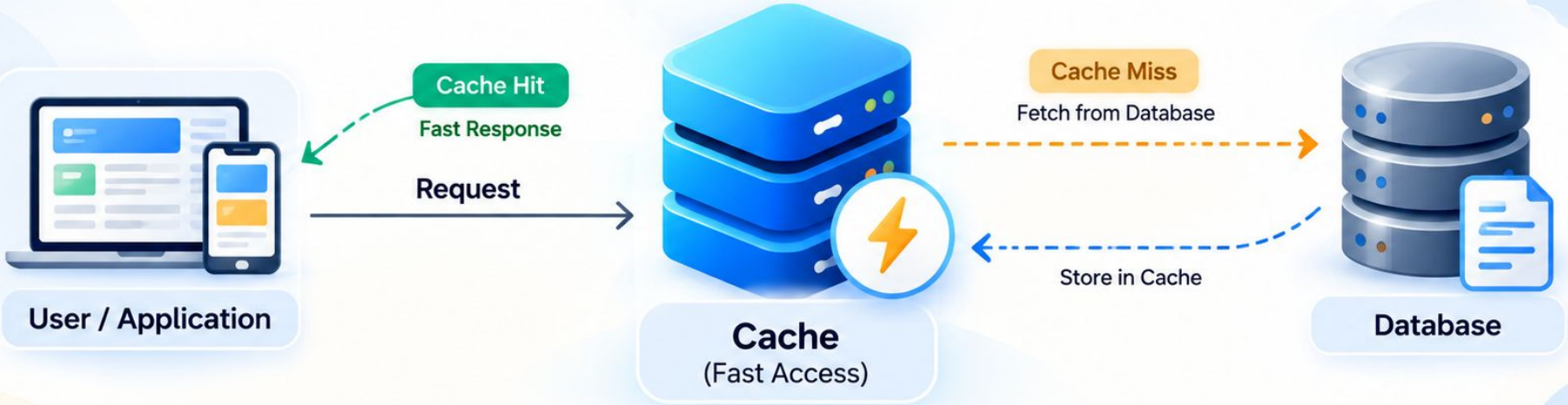


Caching for Speed Optimization

Caching for Speed Optimization

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Why Caching Matters

Reduces latency

by avoiding expensive recomputation or data retrieval.

Eases load

on backend services and databases.

Improves user experience

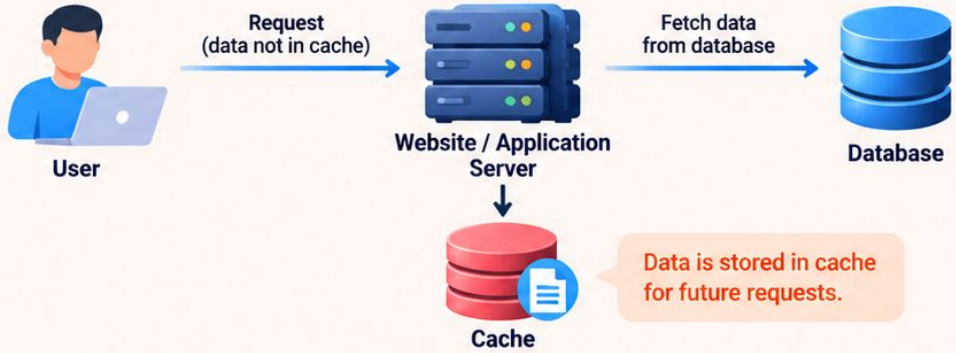
and system scalability.

Critical

in low-latency, high-throughput architectures.

1 First Request

Data is fetched from backend and stored in cache.



2 Subsequent Requests

Data is served from cache (faster).



Types of Caching

1. 

Client-side caching

Browser memory: localStorage, service workers

2. 

Server-side caching

Application-level memory and in-memory caches such as Redis

3. 

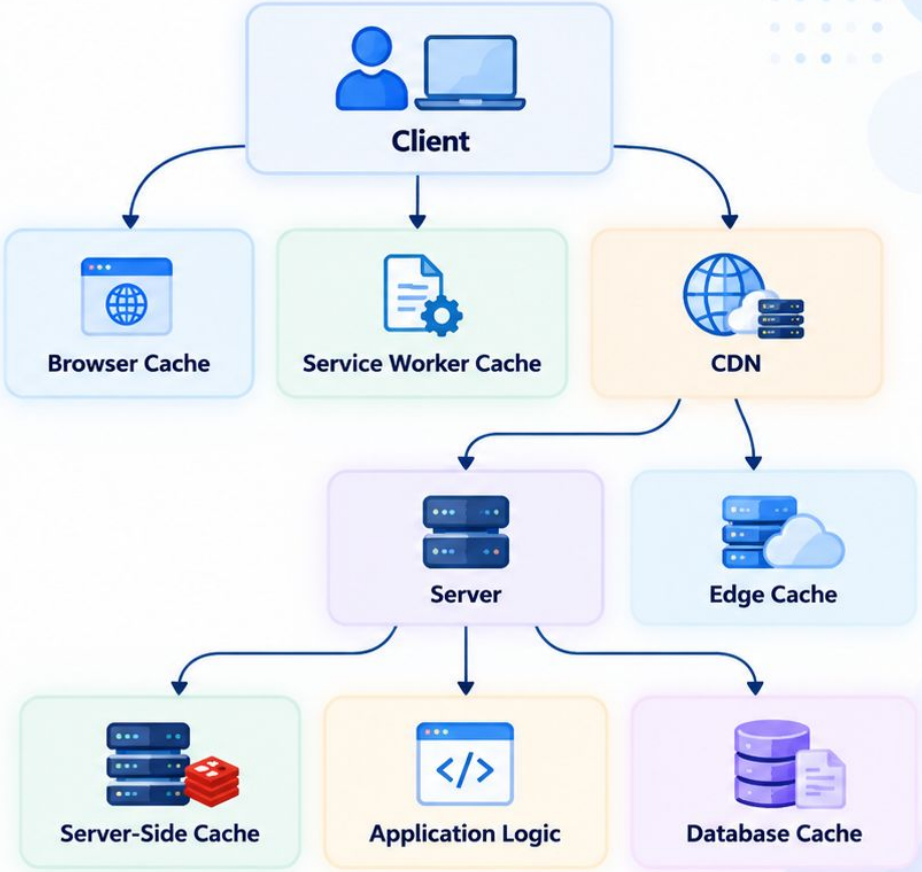
CDN caching

Static content stored closer to users; examples: Cloudflare, Akamai

4. 

Database caching

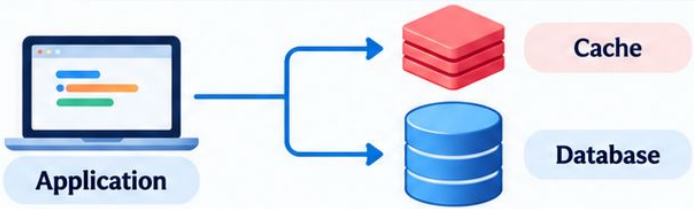
Cached query results and materialized views



Caching Strategies

1 Write-through

Write to cache and database at the same time.



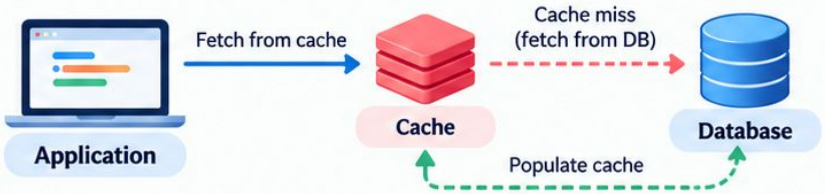
2 Write-back (write-behind)

Write to cache first; update the database asynchronously.



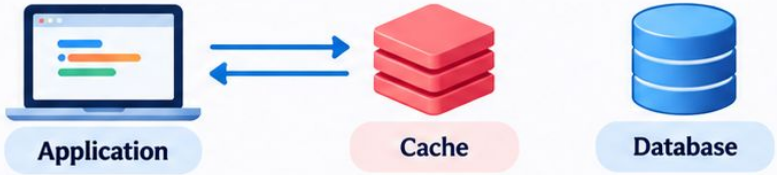
3 Cache-aside (lazy loading)

Fetch from the database on a cache miss, then populate the cache.



4 Explicit / manual caching

The application decides when to cache or evict it.



Cache Eviction Policies

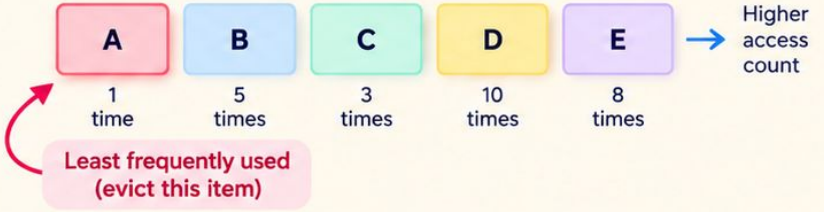
1 LRU (Least Recently Used)

Evict the item that has not been accessed for the longest time.



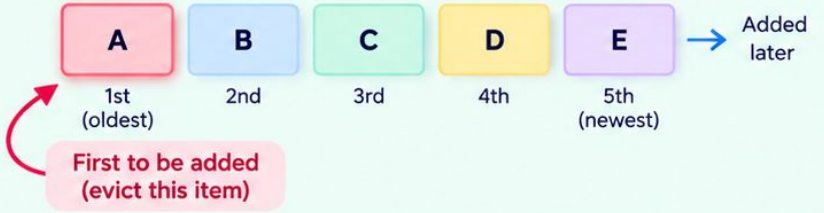
2 LFU (Least Frequently Used)

Evict the item with the fewest accesses.



3 FIFO (First In, First Out)

Evict the item that entered the cache first.



4 TTL (Time To Live)

Expire items automatically after a configured duration.



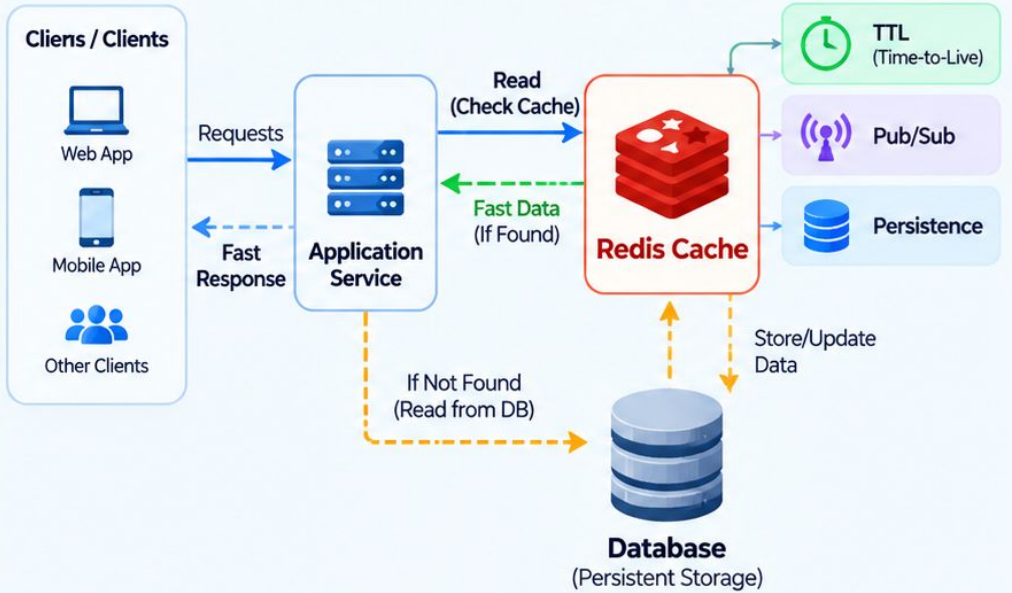
Redis Overview

1. **In-memory key-value store** for ultra-fast access.
2. Supports **TTLs (time-to-live)**, pub/sub, and persistence.
3. Used for **caching, queues, sessions, leaderboards, etc.**
4. **Open-source** and widely adopted.

Common Use Cases

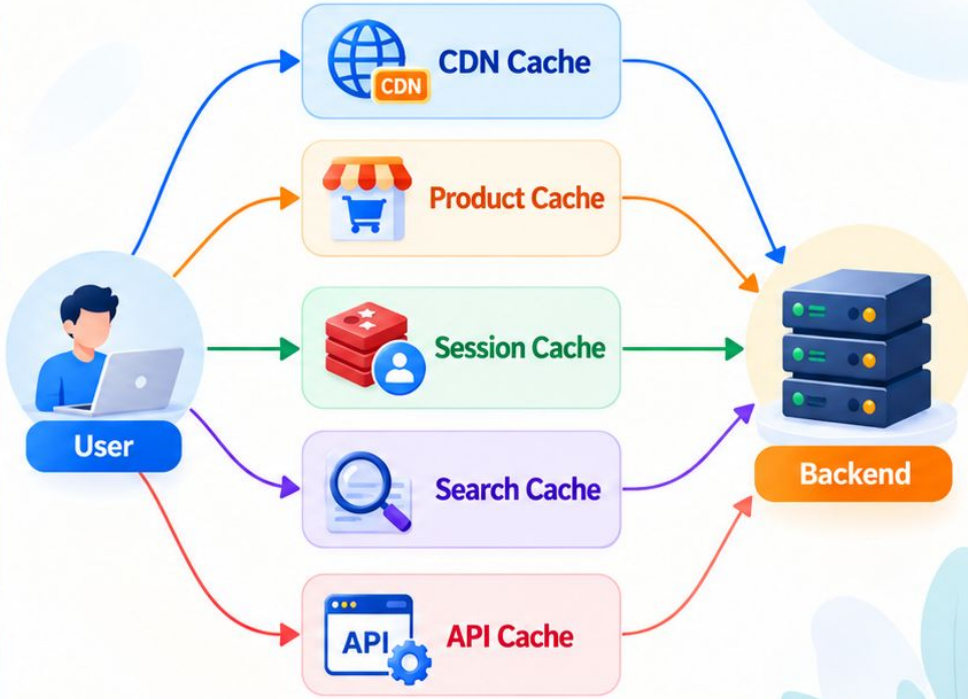
- Caching
- Sessions
- Queues
- Leaderboards

Caching with Redis Cache



Real-World Caching Examples

1. **CDN**
   Cache static assets (images, JS, CSS).
2. **Product page data**
   Cache popular catalog queries.
3. **User sessions**
   Stored in Redis for fast access.
4. **Search results**
   Frequently repeated queries cached.
5. **API response caching**
   Microservices avoid recomputation.



Interview Questions – Caching in System Design

Core Conceptual Questions

1. What is caching and why is it important in system design?
2. Explain different types of caching and where they are used.
3. What are write-through vs. write-back caching strategies?
4. What is lazy loading (cache-aside pattern) and when would you use it?

Scenario-Based Questions

1. How would you use caching to optimize a product details page?
2. What eviction strategy would you choose for a memory-limited system?
3. How would you keep cache and database in sync?
4. What are the potential downsides or risks of aggressive caching?

Practical Implementation

1. How would you implement Redis caching in a web application?
2. How can you prevent cache stampedes or thundering herd problems?
3. What tools can be used for distributed caching?

Instructor: Rahul Rajat Singh

Summary & What's Next



Caching improves **response time**, **reduces load**, and **scales better**.



Choose **cache type**, **strategy**, and **eviction policy** wisely.



Redis is a powerful tool in the caching toolkit.



Caching is foundational in **high-performance systems**.



WHAT'S NEXT

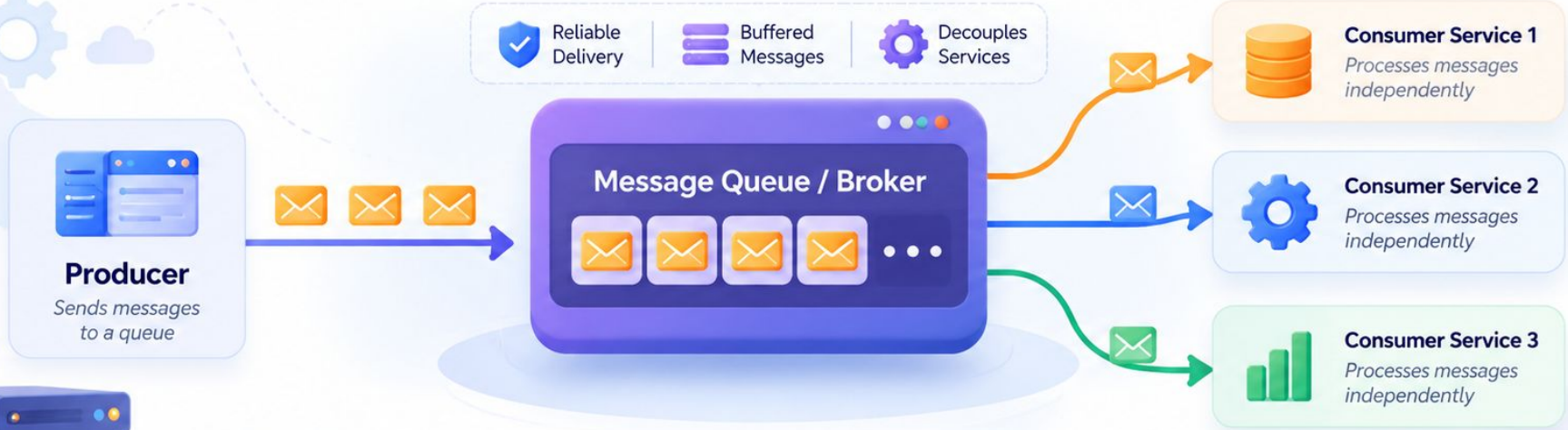
Messaging & Queues for Decoupling



Messaging & Queues for Decoupling

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Why Use Asynchronous Messaging?

Loose Coupling
Decouple producers from consumers.

Improved Performance
Producers don't block waiting for consumers.

Scalability
Consumers can scale independently.

Resilience
Message durability adds fault tolerance.

Flexibility
Easily add new consumers without changing producers.



Key Concepts of Messaging Systems

1. **Message**
   A data packet sent from producer to consumer.

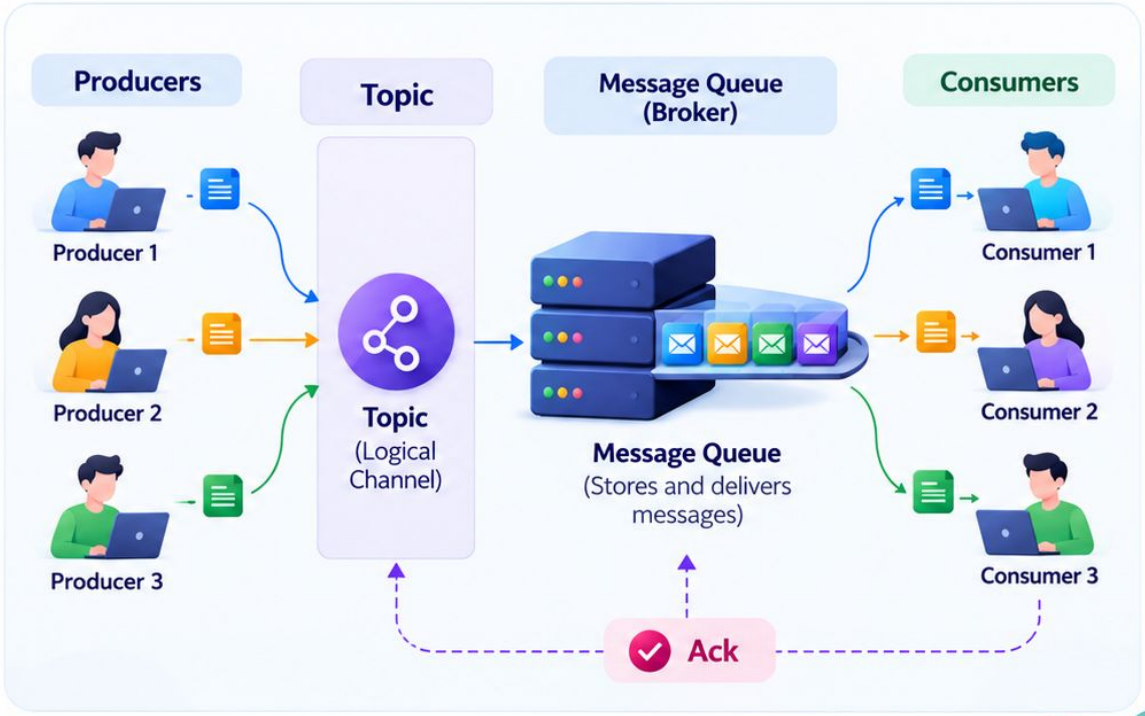
2. **Producer**
   Sends the message.

3. **Consumer**
   Receives and processes the message.

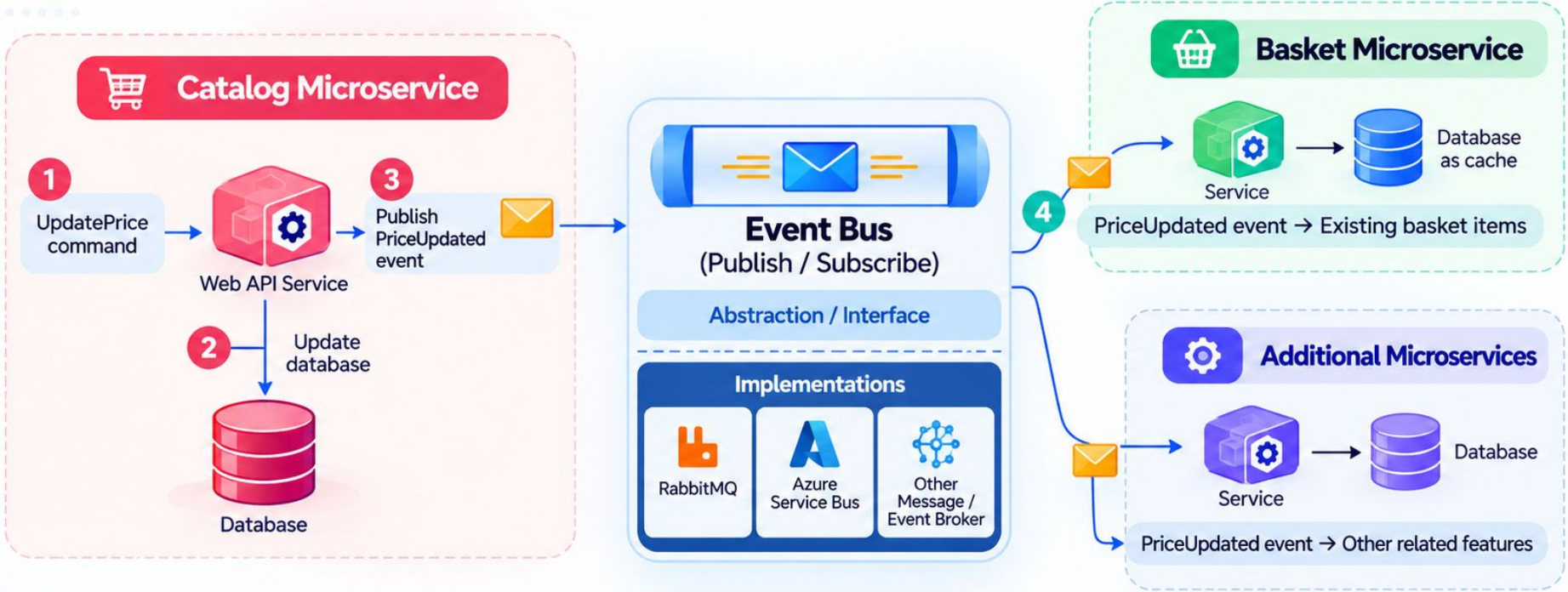
4. **Broker/Queue**
   Stores and delivers messages.

5. **Topic/Queue**
   Logical channel for message delivery.

6. **Ack**
   Acknowledgement of successful processing.



Visualizing a Decoupled Architecture



When to Use Queues in Architecture

1



Burst Workloads

Absorb sudden demand spikes

2



Service Decoupling

Let services work independently

3



Background Jobs

Email, processing, exports

4



Rate-Limited or Expensive Operations

Control when work is performed

5



Traffic Buffering

Smooth peaks before workers process tasks



Delivery Guarantees

1 At-least-once
(default in many systems)



- **Message is retried** until acknowledged.
- **May lead to duplicates.**
- **Consumers must be** idempotent.

2 At-most-once



- **Message sent** only once.
- **No retries** → may result in message loss.

3 Exactly-once



- **Guaranteed single delivery** without duplicates.
- **Complex and** more resource-heavy.
- **Kafka supports it under** specific constraints.

Common Use Cases of Messaging Queues

1 Order Processing

Decouple frontend from inventory, payments, and shipping.



2 Logging & Monitoring

Centralize logs for analysis (e.g., ELK, Kafka).



3 Rate Limiting & Traffic Shaping

Queue incoming requests and process them at a safe pace.





4 Email & SMS Notifications

Queue notifications for asynchronous delivery.



5 ETL Pipelines & Stream Processing

Kafka enables real-time data transformation.



Best Practices for Using Messaging Queues

1 Use idempotent consumers

Ensure that processing the same message multiple times produces the same result.



2 Implement dead-letter queues (DLQs)

Send failed or unprocesable messages to a dead-letter queue (DLQ) for later inspection and handling.



3 Monitor queue length & processing time

Track key metrics like queue length, processing time, and consumer lag to detect issues early.



4 Handle retries & failures gracefully

Implement retry mechanisms with backoff and handle failures without duplicating work.



5 Choose delivery semantics based on need

Select the right delivery semantics (e.g., at-most-once, at-least-once, exactly-once) based on your application requirements.



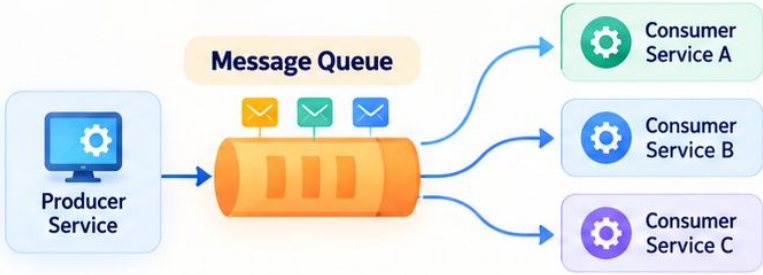
6 Secure your message brokers

Use authentication, encryption, and proper access controls to protect your message brokers.



Instructor: Rahul Rajat Singh

Interview Questions –
Messaging & Queues for Decoupling



FOUNDATIONS

1. Why would you use asynchronous messaging in a system?
2. What are the differences between RabbitMQ and Kafka? When would you use one over the other?

DELIVERY & RELIABILITY

3. Explain the different message delivery guarantees: At-least-once vs. Exactly-once vs. At-most-once – what are they and where do you apply them?
4. How do queues help improve system scalability and fault tolerance?

DESIGN UNDER LOAD

5. What problems might arise in a queue-based system under heavy load? How would you mitigate them?
6. How would you design an order processing system using a message queue?
7. How would you ensure idempotency in the consumers?

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

1



Asynchronous messaging helps build **scalable, decoupled systems.**

2



Use queues when **real-time response isn't required.**

3



RabbitMQ is great for **traditional message queues.**

4



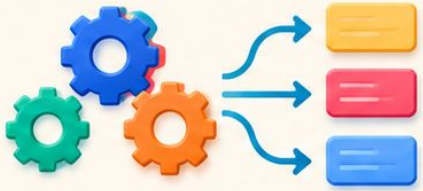
Kafka excels at **high-throughput event streaming.**

5



Understand your **delivery guarantees** to match each use case.

What's Next?



Concurrency & Parallelism

Instructor: Rahul Rajat Singh

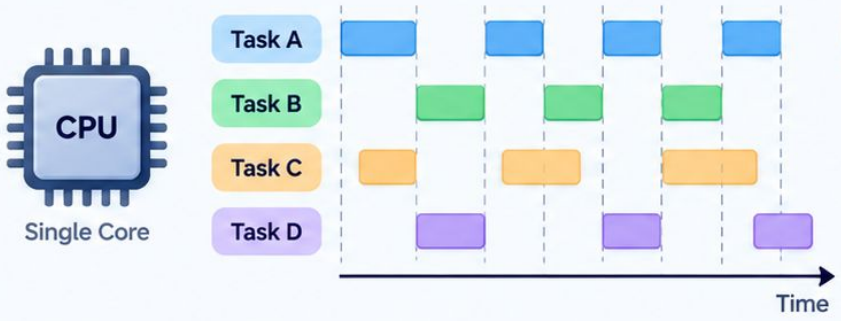
Concurrency & Parallelism

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews

Concurrency

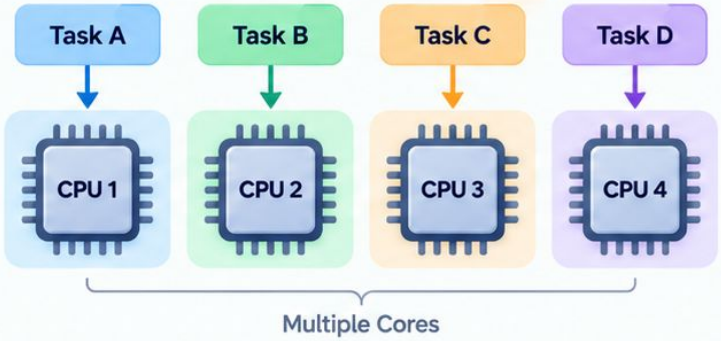
Multiple tasks make progress by interleaving on a **single core** (time sharing).



VS

Parallelism

Multiple tasks run simultaneously on **multiple cores**.



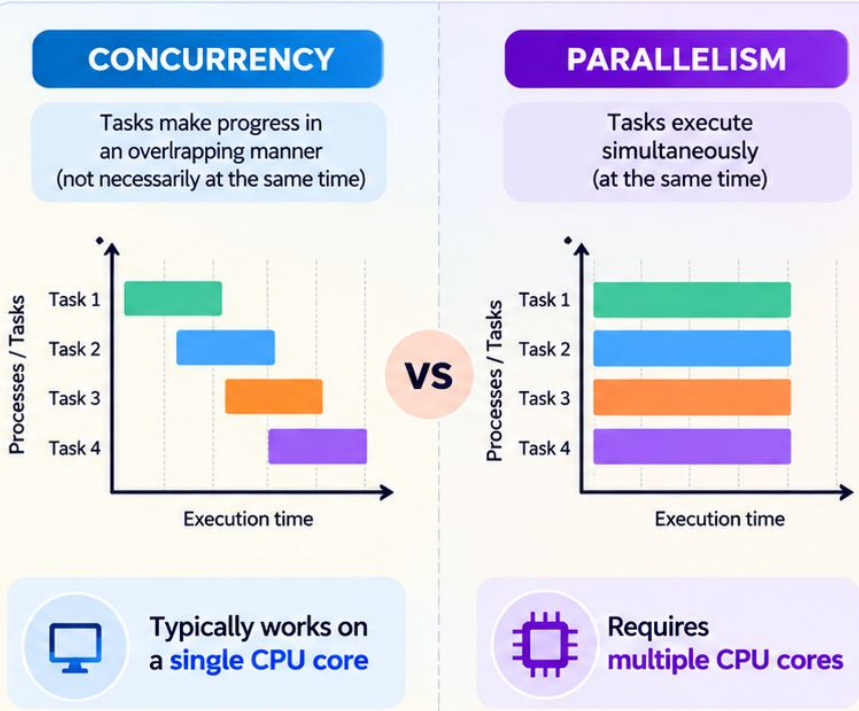
Concurrency and Parallelism

What is Concurrency?

- **Definition:** Concurrency is when multiple tasks start, run, and complete in overlapping time periods – not necessarily simultaneously.
- **Key Points:**
  - It's about managing multiple tasks efficiently, especially on a single CPU core.
  - It's all about task management.
  - Can happen on a single-core.
  - Goal is responsiveness.
- **Example:** Web server handling multiple incoming HTTP requests using asynchronous I/O.

What is Parallelism?

- **Definition:** Parallelism is when multiple tasks are executed simultaneously, typically on multiple CPU cores.
- **Key Points:**
  - It's about actual simultaneous execution to speed up performance.
  - It's all about task execution.
  - Needs multi-core for true parallelism.
  - Goal is speed/throughput.
- **Example:** Matrix computation where different parts are calculated in parallel on multiple threads.



Processes vs. Threads

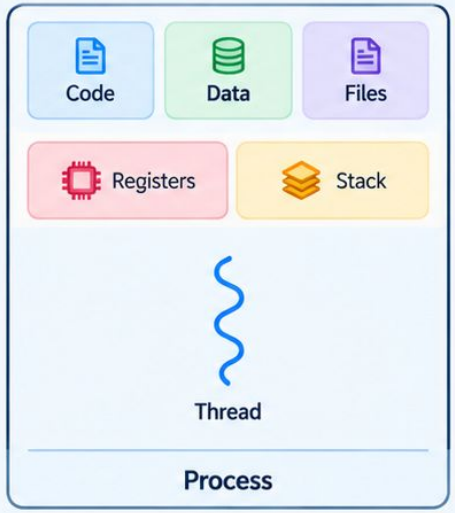
Processes

- Have their own **memory space**
- **Heavier** to create and switch
- **Isolated** and **safer**

Threads

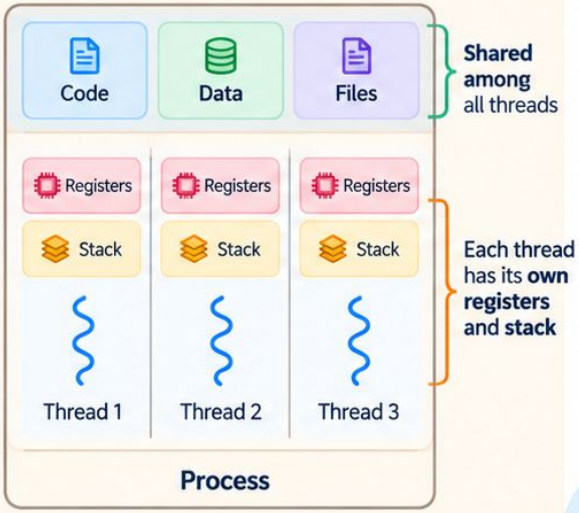
- **Share memory** within a process
- **Lightweight** and **faster**
- Can lead to **complex bugs** (e.g., race conditions)

Single-threaded Process
(One Process, One Thread)



Each process has its **own memory space.**

Multithreaded Process
(Multiple Threads in One Process)



Threads **share memory** within a process (code, data, files).

Thread Pools & Worker Models

1

Thread Pools

- Pre-created threads reused for multiple tasks
- Avoid overhead of creating/ destroying threads

2

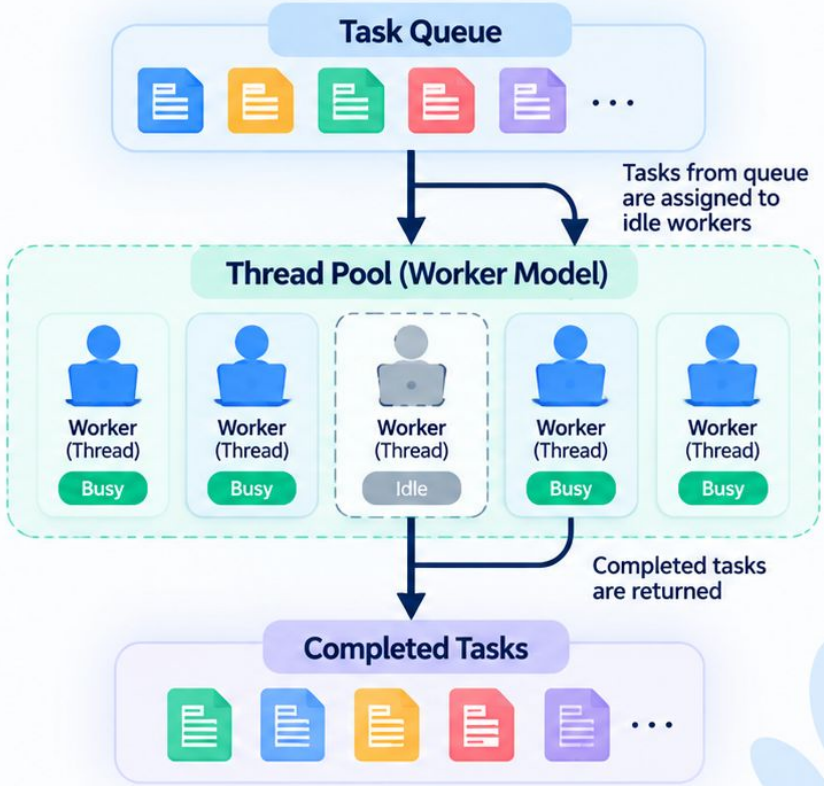
Worker Models

- Tasks are distributed to idle workers from a shared queue
- Improves scalability and CPU utilization

3

Example

- ASP.NET Core uses thread pool to handle requests efficiently.



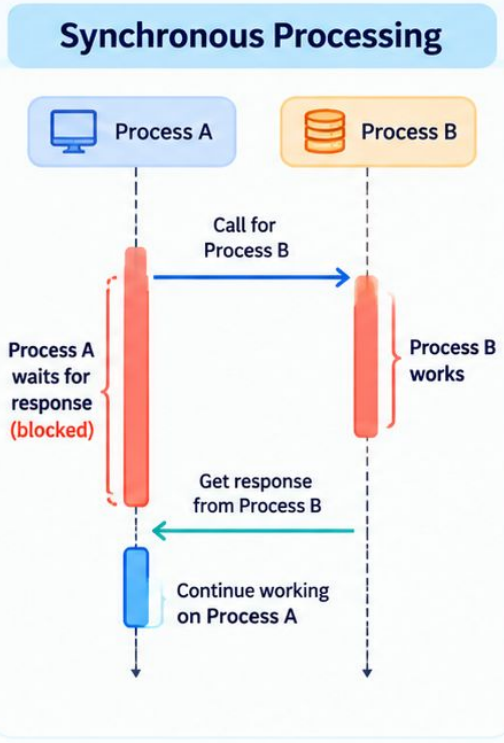
Asynchronous Processing

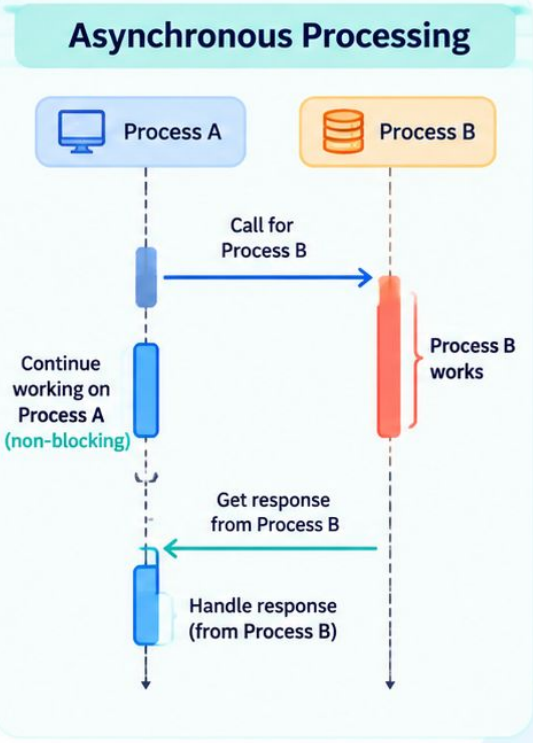
WHY ASYNC?

- Avoid blocking threads on I/O
- Improve throughput

TECHNIQUES

- Async/Await (C#, JavaScript)
- Promises and Futures
- Message queues (RabbitMQ, Kafka) for background work





Concurrency in Web Servers

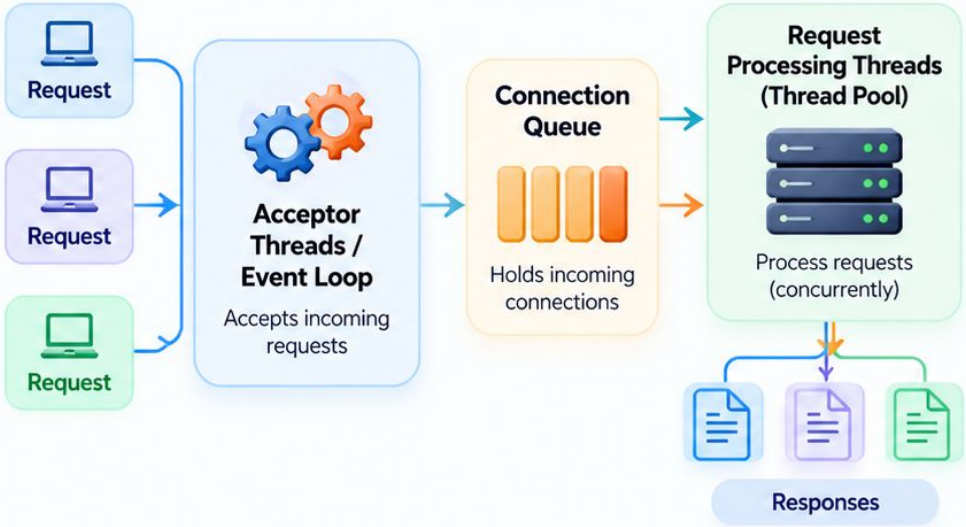
Traditional Servers
(e.g., Apache)

- Spawn a new **thread/process** per request.
- Not scalable for **high traffic.**

Modern Servers
(e.g., Node.js, ASP.NET Core, Nginx)

- Use **async/ non-blocking I/O.**
- Event-loop or thread-pool models for **scalability.**

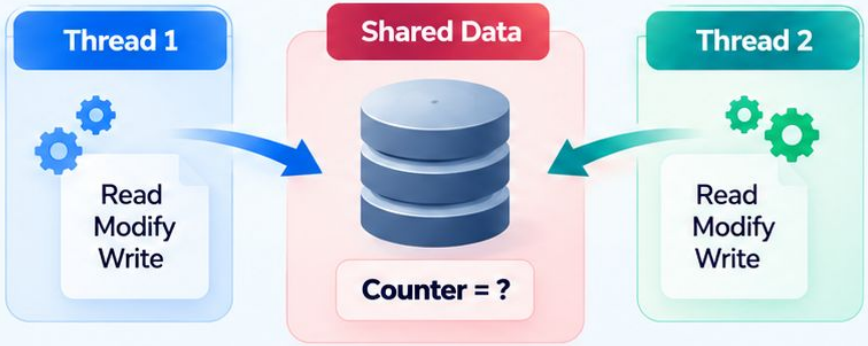
How Concurrency Works in a Web Server



Common Pitfalls

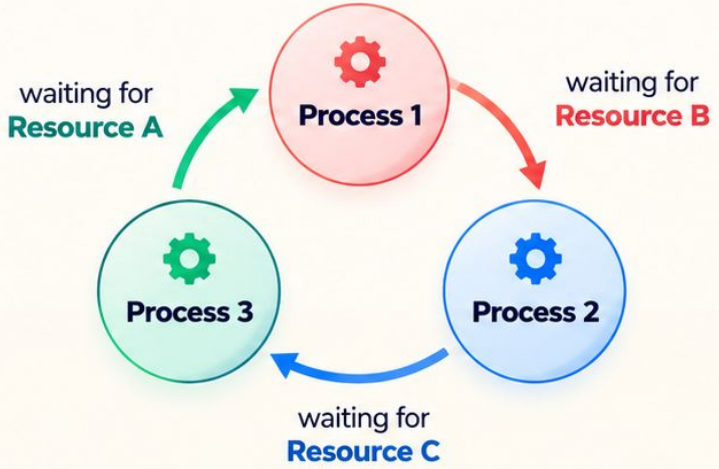
1 Race Conditions

- Multiple threads access and modify shared data concurrently.
- Causes unexpected behavior or data corruption.



2 Deadlocks

- Two or more threads wait for each other to release resources.
- System gets stuck indefinitely.



Best Practices & Real-World Examples

Best Practices

1 Async / non-blocking I/O

Prefer async/non-blocking I/O for I/O-bound tasks.



2 Thread pools

Use thread pools instead of creating raw threads.



3 Synchronize shared data

Always synchronize access to shared data (locks, mutexes, etc.).



Locks
Mutexes
Synchronized Access

4 Prevent deadlocks

Detect and avoid deadlocks (lock ordering, timeout strategies).



Real-World Examples

1 Web Server Handling 1000s of Requests

Uses thread pool + async I/O to handle thousands of requests.



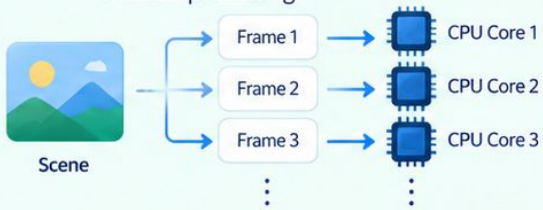
2 Background Job Processing

Worker model with RabbitMQ for email queues.

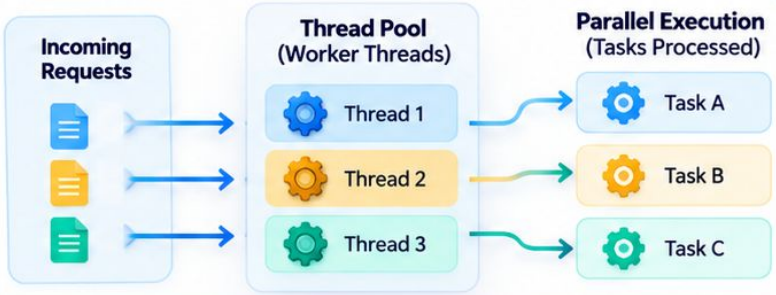


3 Parallel Image Rendering

Each frame rendered on a different core for faster processing.



Interview Questions – Concurrency & Parallelism



CONCEPTUAL QUESTIONS

1. What is the difference between concurrency and parallelism?
2. How do threads differ from processes?
3. What is a thread pool, and why is it preferred over creating new threads?

PRACTICAL SCENARIOS

1. How would you design a web server to handle thousands of concurrent requests?
2. Describe how you would implement background job processing in a scalable system.
3. How would you debug and resolve a deadlock in a multithreaded application?

PITFALL AWARENESS

1. What is a race condition, and how can you prevent it?
2. How do you ensure thread-safe operations in a shared-memory environment?

BONUS (ADVANCED)

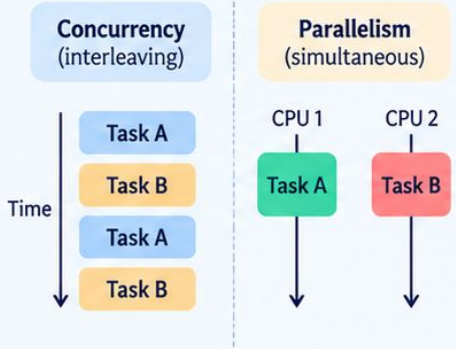
1. How does the event loop work in Node.js or similar environments?
2. What's the difference between parallelism using threads vs. async I/O?

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

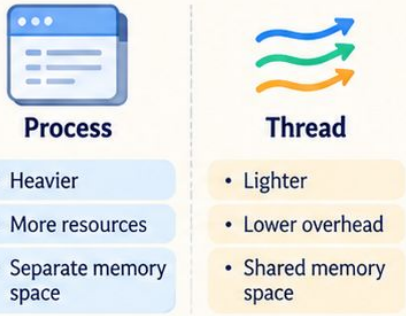
1 Concurrency ≠ Parallelism

Both are tools for performance and scalability.

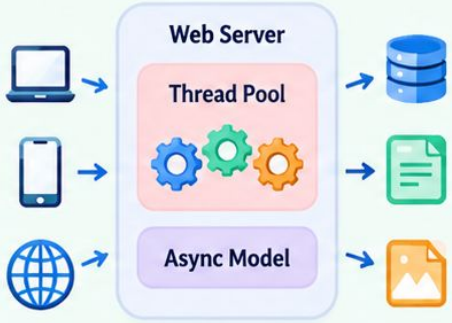


2 Threads are lighter than processes

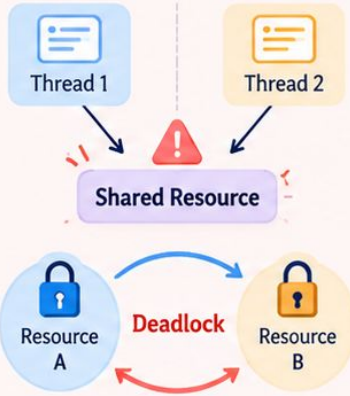
Threads are lighter than processes but need careful handling.



3 Web servers rely on thread pools and async models



4 Watch out for race conditions and deadlocks!



WHAT'S NEXT?

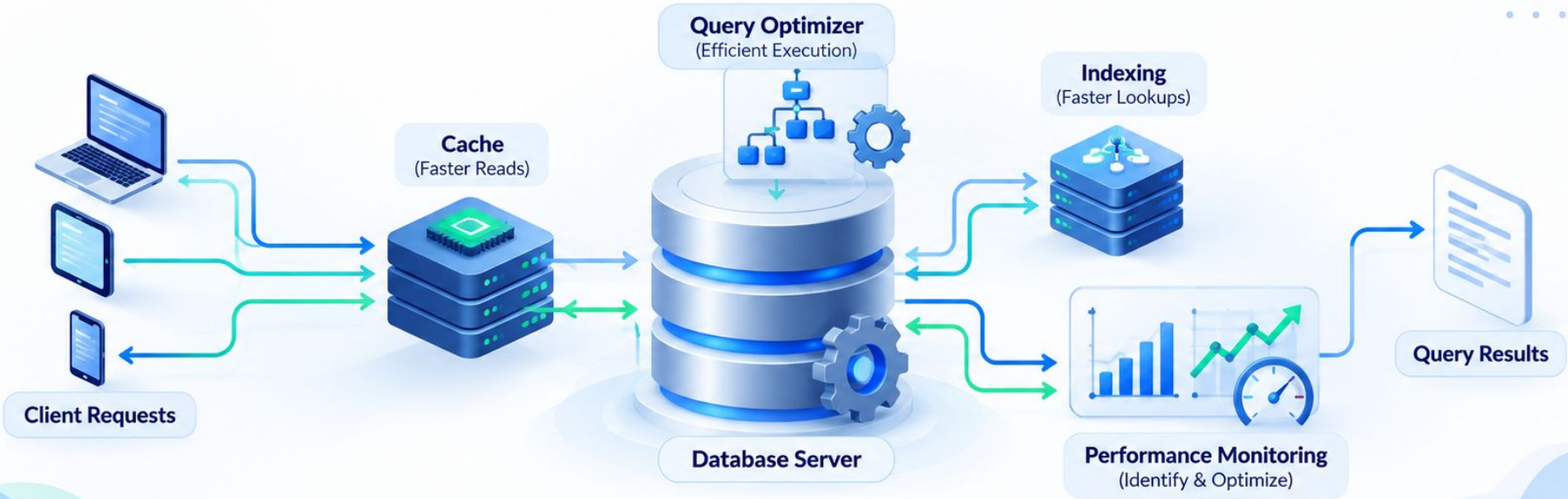
Database Performance Optimization Techniques

Instructor: Rahul Rajat Singh

Database Performance Optimization Techniques

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Replication Recap

01

What is Replication?

Replication is the process of copying and maintaining database objects across multiple databases.

02

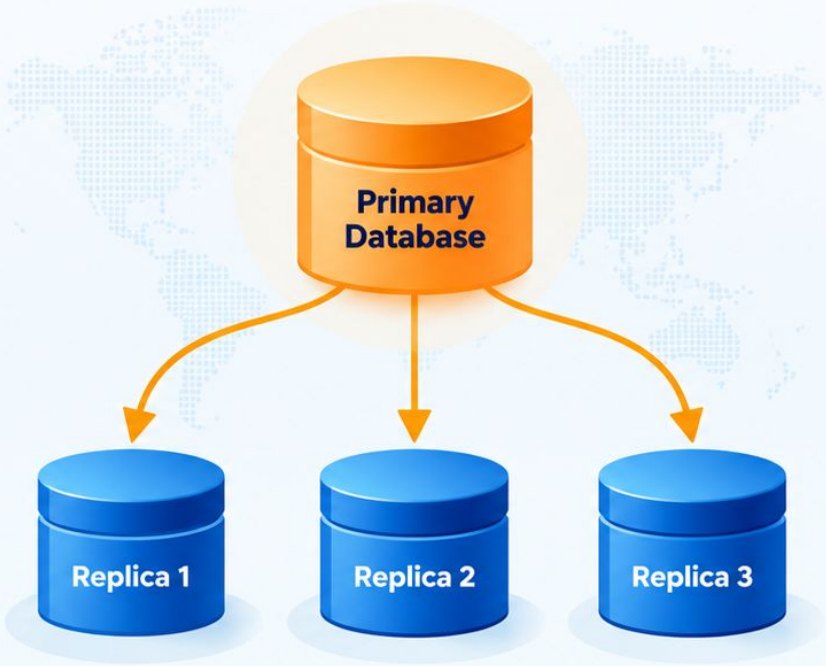
Why Replicate?

- **High Availability:** Ensure data availability in case of failure (data remains accessible).
- **Load Balancing:** Distribute read traffic across multiple servers.
- **Disaster Recovery:** Maintain copies of data across different locations.

03

Types of Replication

- **Primary–Replica Replication:** One primary database, multiple replicas for reads.
- **Multi-Primary Replication:** Multiple primary databases, often used for writes and redundancy.



Sharding & Partitioning Strategies Recap

1 Sharding

Definition: Splitting large datasets into smaller, more manageable pieces (shards).

Goal: Distribute data across multiple servers for better performance and scalability.

Example: User data across multiple shards based on geographical region.



2 Partitioning

Definition: Dividing data within a single database into separate tables or partitions.

Types of Partitioning:

Range Partitioning
Split data by a range (e.g., by date).
Example: Sales data by year/month.

Hash Partitioning
Split data by hashing on a key.
Example: User data by user_id (hash).

Vertical Partitioning (Split Columns)

Split columns/fields into separate tables.

Original Table (All Fields)

CustomerID	Name	Email	Total Purchases
1	ABC	abc@gmail.com	2000
2	PQR	pqr@gmail.com	3000
3	XYZ	xyz@gmail.com	4000
4	LMN	lmn@gmail.com	5000
5	JKL	jkl@gmail.com	6000

Customer Info (Basic Fields)

CustomerID	Name	Email
1	ABC	abc@gmail.com
2	PQR	pqr@gmail.com
3	XYZ	xyz@gmail.com
4	LMN	lmn@gmail.com
5	JKL	jkl@gmail.com

Purchase Info (Separate Table)

CustomerID	Total Purchases
1	2000
2	3000
3	4000
4	5000
5	6000

Horizontal Partition / Database Sharding (Split Rows)

Split rows/records across multiple shards or partitions.

Original Table (All Records)

CustomerID	Name	Email	Total Purchases
1	ABC	abc@gmail.com	2000
2	PQR	pqr@gmail.com	3000
3	XYZ	xyz@gmail.com	4000
4	LMN	lmn@gmail.com	5000
5	JKL	jkl@gmail.com	6000

Shard 1 (e.g., Users from A–M)

CustomerID	Name	Email	Total Purchases
1	ABC	abc@gmail.com	2000
2	PQR	pqr@gmail.com	3000
3	XYZ	xyz@gmail.com	4000

Shard 2 (e.g., Users from N–Z)

CustomerID	Name	Email	Total Purchases
4	LMN	lmn@gmail.com	5000
5	JKL	jkl@gmail.com	6000

CAP Theorem Recap

Performance Focus & Distributed-System Trade-offs

CONSISTENCY

Reads reflect a single, up-to-date view of data.

AVAILABILITY

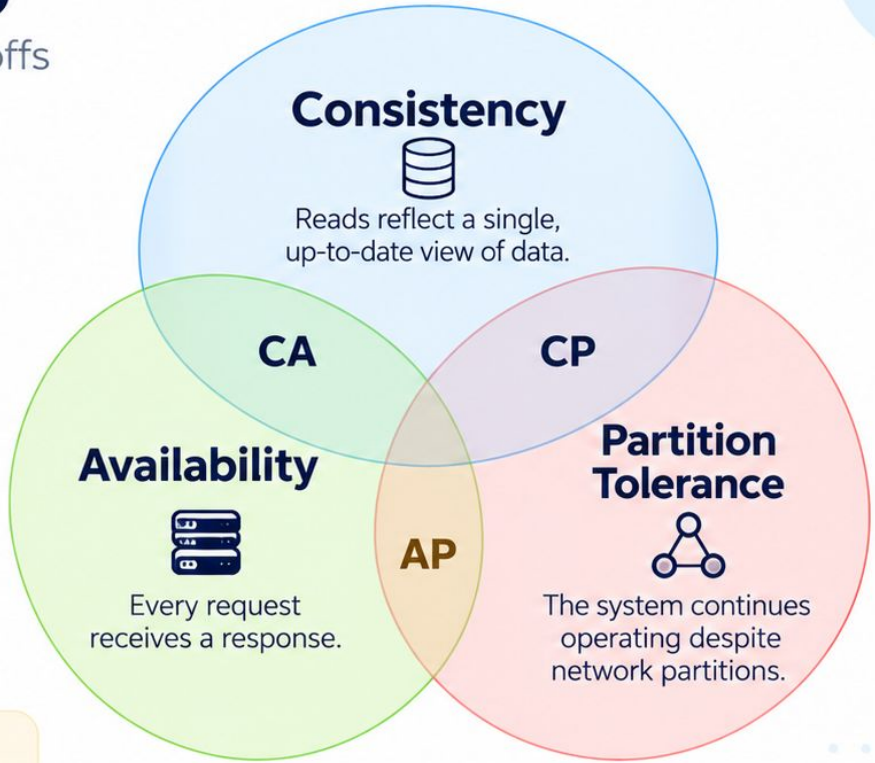
Every request receives a response.

PARTITION TOLERANCE

The system continues operating despite network partitions.

The practical trade-off

When a network partition occurs, a distributed system must prioritize consistency or availability for affected operations.



CAP trade-offs matter during partitions.

Indexes: Types & Use Cases

01 What are Indexes?

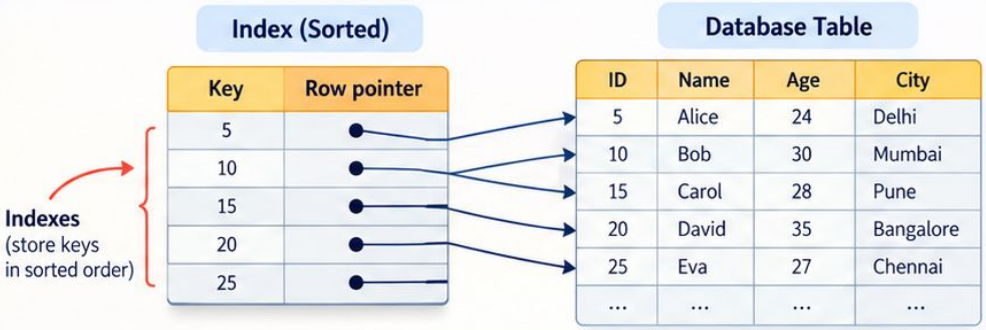
Data structures that speed up queries by reducing the amount of data scanned.

02 Types of Indexes

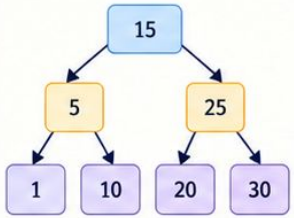
B-Tree	Common for exact matches and range queries.
Hash	Best for equality lookups.
Full-Text	Used for searching large textual data.
Bitmap	Suitable for low-cardinality columns (e.g., gender).

03 When to Use Indexes

- **Read-heavy operations:** Indexes speed up query performance.
- **Write-heavy systems:** Be cautious, as indexes can slow down inserts and updates.

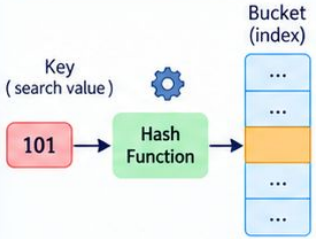


B-Tree Index



Balanced tree structure. Good for exact matches and range queries.

Hash Index



Uses a hash function to find the row. Best for equality lookups.

Bitmap Index

ID	Gender (Bitmap)	
	Male	Female
5	1	0
10	0	1
15	1	0
20	1	0
25	0	1
...	...	...

Uses bitmaps for each value. Suitable for low-cardinality columns (e.g., gender).

Instructor: Rahul Rajat Singh

Normalization vs. Denormalization

Normalization

Organize data into tables to reduce redundancy

Goal	Reduce data redundancy by organizing data into tables.
Benefits	Minimizes storage costs, eliminates anomalies.
Drawback	Can lead to complex joins and slower read performance.

Denormalization

Introduce redundancy to reduce join operations

Goal	Introduce redundancy to reduce join operations and speed up reads.
Benefits	Faster read performance.
Drawback	Increased storage and potential data anomalies.

When to Use Each

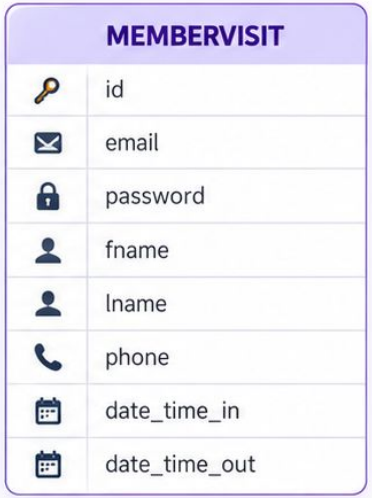
- **Normalization**: For transactional systems (OLTP).
- **Denormalization**: For reporting systems or read-heavy workloads.

Normalized



VS

Denormalized



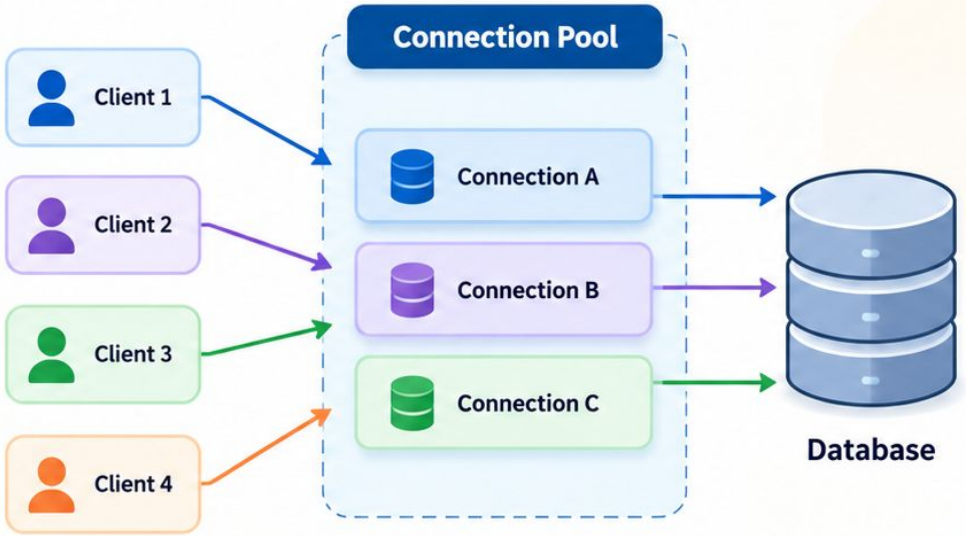
Additional Techniques – Connection Pooling

What is Connection Pooling?

A technique used to manage database connections efficiently by reusing established connections instead of creating new ones.

Why Use It?

- Reduces overhead caused by frequent connection creation and teardown.
- Helps handle a large number of concurrent connections effectively.



Additional Techniques - Query Optimization



Definition

The process of improving the performance of SQL queries.

Techniques

1 Use of Indexes

Leverage indexes to speed up search operations.



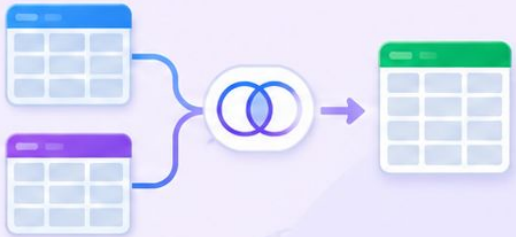
2 Avoiding N+1 Queries

Reduce unnecessary database hits by using joins or batching queries.



3 Using Proper Joins

Minimize complex joins when possible.



Additional Techniques - Materialized Views

Definition

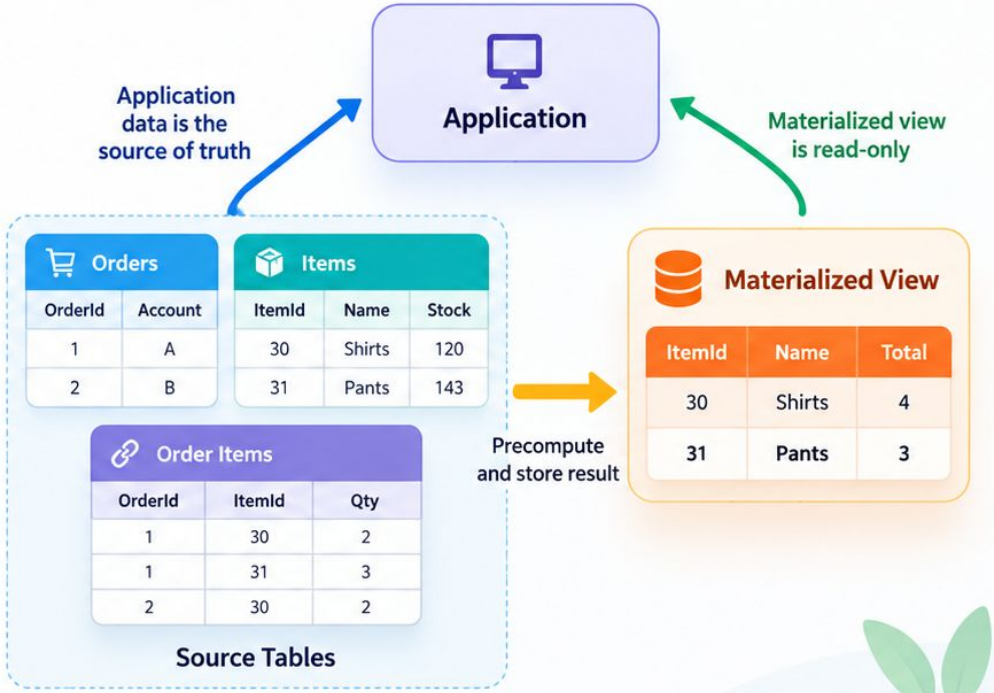
A precomputed query result stored as a table.

Benefits

- Speeds up query performance by avoiding real-time computation.
- Useful in reporting and data warehousing.

Use Cases

- Data aggregation or summary data that doesn't change frequently.
- Reporting systems where fast retrieval is critical.



Additional Techniques – Batching & Pagination

Batching

Definition: Sending multiple operations in a single request or transaction to reduce overhead.

Use Case:
Bulk inserts or updates.



Pagination

Definition: Breaking large sets of data into smaller chunks for efficient retrieval.

Prevents large queries
that could lead to timeouts or memory issues.

Ensures responsive UI
by fetching data incrementally.



Large Dataset
(e.g., 100,000 records)

Interview Questions

Database Performance Optimization Techniques



1. **What is database replication,** and why is it important for performance?
2. **Can you explain the difference** between sharding and partitioning in databases?
3. **How does the CAP theorem impact** database performance decisions?
4. **What are some common types of indexes,** and when would you use them?
5. **What is the difference between normalization and denormalization,** and how do they affect performance?
6. **How does connection pooling improve** database performance?
7. **What is query optimization,** and how would you approach optimizing a slow query?
8. **What are materialized views,** and how do they enhance performance?
9. **How would you handle large datasets** in your application?
10. **What are the trade-offs between consistency and performance** in distributed databases?

Instructor: Rahul Rajat Singh

Summary and Key Takeaways

1 Replication & Sharding

Key techniques for scaling databases across multiple servers.



2 CAP Theorem

Understanding trade-offs between consistency, availability, and partition tolerance.



3 Indexes

Crucial for improving query performance, but need to be managed properly.



4 Normalization vs. Denormalization

Choose based on workload (transactions vs. reporting).



Normalization

Denormalization

5 Other Techniques

Connection pooling, query optimization, materialized views, and batching for efficient database operations.



Connection Pooling

Query Optimization

Materialized Views

Batching

6 What's Next?

Summary and Recap - Performance



Instructor: Rahul Rajat Singh

Section Summary - Performance

1



Introduction to System Performance

2



Caching for Speed Optimization

3



Messaging & Queues for Decoupling

4



Concurrency & Parallelism

5



Database Performance Optimization Techniques

WHAT'S IN NEXT SECTION?

Reliability – Availability, Failover & Recovery

